

SUMMARY

This video explains the advanced construction and optimization principles for Minimal Deterministic Finite Automata (DFA) within Theory of Computation (TOC). The analysis addresses substring detection, parity constraints, compound prefix-length conditions, and structural minimization logic.

Key Takeaways:

The $m + n + 1$ rule determines state count for prefix-plus-modulo constraints.
Languages equating counts of "01" and "10" are regular because their difference is bounded.
Many standard textbook DFA constructions are not minimal.
True minimization requires structural reasoning rather than memorization.

I. SUBSTRING CONSTRAINTS: DEBIT AND TRIBIT

Substring DFAs are built by constructing the shortest acceptance path and preserving partial progress when mismatches occur.

I.1 Debit Substrings ("00" or "11")

Objective: Accept strings containing either "00" or "11".

State Count: 4

Construction Logic:

Track single occurrences of 0 and 1.

If two identical consecutive symbols appear, transition to a final state.

Final state contains self-loops on both inputs (0,1).

I.2 Tribit Substrings ("000" or "111")

State Count: 6

Construction Logic:

Track first, second, and third consecutive identical symbols.

If interruption occurs (for example, "00" followed by "1"), transition to a state tracking the new character rather than resetting fully.

Final state self-loops once the substring condition is satisfied.

II. EQUALITY OF SUBSTRING OCCURRENCES

Language Definition:

Number of occurrences of “01” equals number of occurrences of “10”.

Regularity Insight:

In any binary string, the difference between the count of “01” and “10” is always -1 , 0 , or 1 . Because the difference is bounded, the language is regular.

Language Composition:

Accepts:

ϵ
0
00
1
11
010
101

Rejects:

01
10

Minimal DFA:

State Count: 5

Structure:

Initial state is final (accepts ϵ).

Separate branches manage strings starting with 0 and 1.

Cross-transitions balance substring counts.

III. PARITY AND MODULO CONSTRAINTS

Languages based on parity of substring occurrences follow cyclic constructions.

Language Constraint	States	Logic
"10" occurs Even times	4	Accepts ϵ and zero occurrences
"10" occurs Odd times	4	Complement of Even DFA

The Odd DFA is obtained by swapping final and non-final states of the Even DFA.

IV. COMPOUND CONSTRAINTS: START AND LENGTH DIVISIBILITY

Language Structure:

Strings must start with substring S of length m.

Total length $\equiv r \pmod n$.

IV.1 The $m + n + 1$ Rule

Total States = $m + n + 1$

Where:

m = length of required starting substring

n = modulus of length constraint

1 = dead state

IV.2 Application Examples

Starting Substring	Length Constraint	Calculation	Total States
"a" (m = 1)	Divisible by 3 (n = 3)	$1 + 3 + 1$	5
"ba" (m = 2)	$2 \pmod 4$ (n = 4)	$2 + 4 + 1$	7
"aba" (m = 3)	$2 \pmod 5$ (n = 5)	$3 + 5 + 1$	9

Dead state handles prefix violations.

Remaining states implement modulo-length cycling.

V. POSITIONAL "FOLLOWED BY" CONSTRAINTS

V.1 Every 'A' is Followed by 'B'

Minimal States: 3

States:

1. Final state accepting B-sequences and ϵ
2. Non-final state after reading 'A', waiting for 'B'
3. Dead state if 'A' is followed by another 'A'

V.2 Every 'A' is Never Followed by 'B'

States Required: 3

If substring "AB" appears, transition to dead state.

V.3 Every 'A' is Followed by "BB"

States Required: 4

An additional state is required to track the first 'B' after 'A'.

VI. TECHNICAL CRITIQUE: GATE 2004 AMBIGUITY

Problem Context:

Language defined as:

Every 'A' is followed by at least two 'B's.

Issue:

If 'A' is absent, strings containing only 'B's must be accepted.
The exam diagram did not clearly mark all required final states.

Conclusion:

The diagram was structurally incomplete.
Ambiguity likely resulted in universal marking.

VII. STRATEGIC PRINCIPLES FOR MINIMIZATION

Construct the minimal forward acceptance path first.
Preserve partial progress rather than resetting fully.

Use complement logic for Even/Odd transformations.
Apply the $m + n + 1$ rule for prefix-plus-modulo languages.

